

TD4 : Web Scraping Avancé et Analyse de Données avec Pandas

Scraping + Pandas + Visualisation

Applications Concrètes au Génie Civil

Niveau : École d'Ingénieurs - Première année

Durée : 2h30

Prérequis : TD3 - Introduction au Web Scraping

22 octobre 2025

Table des matières

1	Introduction et rappels	3
1.1	Objectifs du TD4	3
1.2	Rappels du TD3	3
1.3	Nouvelles notions du TD4	3
1.4	Présentation du site MarketBTP	3
1.5	Vérification de l'environnement	4
2	Introduction à Pandas (20 min)	4
2.1	Qu'est-ce que Pandas ?	4
2.2	Structure principale : le DataFrame	4
2.3	Opérations de base sur les DataFrames	4
2.3.1	Affichage et information	4
2.3.2	Sélection de données	5
2.3.3	Calculs statistiques	5
2.3.4	Tri et classement	5
2.3.5	Export de données	6
2.4	Visualisation avec Matplotlib	6
3	Exercices pratiques (1h20)	6
3.1	Exercice 2 : Scraper et analyser le catalogue MarketBTP (50 min)	6
3.1.1	Objectifs	6
3.1.2	Code de départ	7
3.1.3	Indices pour l'extraction	9
3.2	Exercice 3 : Analyse comparative avancée (30 min)	10
3.2.1	Objectifs	10
3.2.2	Code de départ	11
4	Cas d'usage réels en Génie Civil avec Pandas	11
4.1	Analyse de prix de matériaux	11
4.2	Analyse d'appels d'offres	12
5	Ressources complémentaires	12
5.1	Documentation officielle	12
5.2	Tutoriels Pandas recommandés	13
6	Points clés à retenir	13
6.1	Scraping	13
6.2	Pandas	13
6.3	Analyse de données	13
7	Préparation pour le TP2	14

1 Introduction et rappels

1.1 Objectifs du TD4

Ce quatrième TD se concentre sur des techniques avancées de web scraping et l'analyse des données collectées avec Pandas. Vous allez :

- Scraper un catalogue en ligne de matériaux BTP (MarketBTP)
- Gérer la pagination pour collecter de grandes quantités de données
- **Utiliser Pandas pour analyser les données collectées**
- **Réaliser des statistiques descriptives sur des données de génie civil**
- **Visualiser les données avec Matplotlib**
- Comparer les prix et identifier les meilleures opportunités

1.2 Rappels du TD3

Lors du TD3, vous avez appris :

- La structure HTML et les sélecteurs CSS
- L'utilisation de BeautifulSoup pour parser le HTML
- L'extraction de données depuis des fichiers locaux
- Les principes éthiques du web scraping

1.3 Nouvelles notions du TD4

Ce TD introduit :

- Les requêtes HTTP vers des sites en ligne
- La gestion de la pagination multi-pages
- **L'utilisation de Pandas pour manipuler et analyser des données**
- **Les DataFrames : structures de données tabulaires**
- **Statistiques descriptives : moyenne, médiane, min, max, écart-type**
- **Filtrage et tri de données**
- **Visualisation de données avec Matplotlib**
- Les bonnes pratiques (délais, gestion d'erreurs)

1.4 Présentation du site MarketBTP

Pour ce TD, vous allez scraper un site web en ligne : **MarketBTP**, un catalogue professionnel de matériaux de construction.

URL du site : <http://www.malomatique.free.fr/MarketBTP/index.html>

Caractéristiques du site :

- 60 produits BTP répartis sur 3 pages
- 5 catégories : Béton, Acier, Maçonnerie, Isolation, Charpente
- Pour chaque produit : nom, fournisseur, prix, caractéristiques techniques, délai, région, note, disponibilité
- Données réalistes pour analyses pertinentes
- Pagination fonctionnelle : page-2.html, page-3.html

1.5 Vérification de l'environnement

Assurez-vous d'avoir installé les bibliothèques nécessaires :

```
1 pip install requests beautifulsoup4 lxml pandas matplotlib
```

Test rapide :

```
1 import requests
2 from bs4 import BeautifulSoup
3 import pandas as pd
4 import matplotlib.pyplot as plt
5
6 print("Environnement pret !")
7 print(f"Version Pandas : {pd.__version__}")
```

2 Introduction à Pandas (20 min)

2.1 Qu'est-ce que Pandas ?

Pandas est une bibliothèque Python puissante pour la manipulation et l'analyse de données. Elle est particulièrement utile pour :

- Organiser des données tabulaires (comme des tableaux Excel)
- Calculer des statistiques (moyennes, totaux, etc.)
- Filtrer et trier des données
- Exporter vers CSV, Excel, etc.
- Visualiser des données

2.2 Structure principale : le DataFrame

Un DataFrame est un tableau à deux dimensions avec des lignes et des colonnes.

```
1 import pandas as pd
2
3 # A partir d'un dictionnaire
4 data = {
5     'Materiau': ['Beton', 'Acier', 'Bois'],
6     'Prix': [95.50, 850.00, 120.00],
7     'Unite': ['m3', 'tonne', 'm3']
8 }
9
10 df = pd.DataFrame(data)
11 print(df)
12
13 # Resultat :
14 #   Materiau  Prix  Unite
15 # 0   Beton   95.50    m3
16 # 1   Acier  850.00  tonne
17 # 2   Bois  120.00    m3
```

Listing 1 – Créer un DataFrame

2.3 Opérations de base sur les DataFrames

2.3.1 Affichage et information

```
1 # Afficher les premières lignes
2 print(df.head())
3
4 # Afficher les dernières lignes
5 print(df.tail())
6
7 # Informations sur le DataFrame
8 print(df.info())
9
10 # Statistiques descriptives
11 print(df.describe())
12
13 # Dimensions (lignes, colonnes)
14 print(df.shape)
```

Listing 2 – Explorer un DataFrame

2.3.2 Sélection de données

```
1 # Sélectionner une colonne
2 prix = df['Prix']
3
4 # Sélectionner plusieurs colonnes
5 selection = df[['Materiau', 'Prix']]
6
7 # Sélectionner des lignes par index
8 premiere_ligne = df.iloc[0]
9
10 # Filtrer des lignes selon une condition
11 matériaux_chers = df[df['Prix'] > 100]
```

Listing 3 – Sélectionner des données

2.3.3 Calculs statistiques

```
1 # Moyenne
2 prix_moyen = df['Prix'].mean()
3
4 # Mediane
5 prix_median = df['Prix'].median()
6
7 # Minimum et maximum
8 prix_min = df['Prix'].min()
9 prix_max = df['Prix'].max()
10
11 # Somme
12 prix_total = df['Prix'].sum()
13
14 # Ecart-type
15 ecart_type = df['Prix'].std()
16
17 print(f"Prix moyen : {prix_moyen:.2f}")
18 print(f"Prix total : {prix_total:.2f}")
```

Listing 4 – Statistiques sur les données

2.3.4 Tri et classement

```

1 # Trier par prix croissant
2 df_trie = df.sort_values('Prix')
3
4 # Trier par prix décroissant
5 df_trie_desc = df.sort_values('Prix', ascending=False)
6
7 # Trier par plusieurs colonnes
8 df_multi = df.sort_values(['Unite', 'Prix'])

```

Listing 5 – Trier les donnees

2.3.5 Export de données

```

1 # Exporter en CSV
2 df.to_csv('materiaux.csv', index=False, encoding='utf-8')
3
4 # Exporter en Excel (necessite openpyxl)
5 df.to_excel('materiaux.xlsx', index=False)
6
7 # Lire un CSV
8 df_lecture = pd.read_csv('materiaux.csv')

```

Listing 6 – Exporter un DataFrame

2.4 Visualisation avec Matplotlib

```

1 import matplotlib.pyplot as plt
2
3 # Configuration pour affichage correct des accents
4 plt.rcParams['font.sans-serif'] = ['DejaVu Sans']
5
6 # Graphique en barres
7 df.plot(x='Materiau', y='Prix', kind='bar',
8         title='Prix des materiaux',
9         xlabel='Materiau',
10        ylabel='Prix (euros)')
11 plt.xticks(rotation=45)
12 plt.tight_layout()
13 plt.savefig('prix_materiaux.png')
14 plt.show()
15
16 # Diagramme circulaire
17 df.set_index('Materiau')['Prix'].plot(kind='pie',
18                                     autopct='%1.1f%%')
19 plt.title('Repartition des couts')
20 plt.show()

```

Listing 7 – Graphiques simples

3 Exercices pratiques (1h40)

3.1 Exercice 1 : Scraper et analyser le catalogue MarketBTP (60 min)

Site à scraper : <http://www.malomatique.free.fr/MarketBTP/index.html>

3.1.1 Objectifs

Créez un script `exercice1.py` qui :

1. Scrape les 3 pages du catalogue MarketBTP
2. Extrait pour chaque produit :
 - Le type de produit (Béton, Acier, Maçonnerie, etc.)
 - Le nom du produit
 - Le fournisseur
 - Le prix (valeur numérique)
 - L'unité (m³, tonne, m², ml, unité)
 - Les caractéristiques techniques
 - Le délai de livraison
 - La région
 - La note (nombre d'étoiles)
 - La disponibilité
3. Stocke les données dans un DataFrame Pandas
4. Effectue une analyse complète :
 - Affiche les statistiques descriptives
 - Identifie les produits les plus chers et les moins chers
 - Calcule le prix moyen par catégorie
 - Calcule le prix moyen par fournisseur
 - Analyse les délais de livraison
 - Filtre les produits disponibles uniquement
5. Crée des visualisations :
 - Graphique en barres des 10 produits les plus chers
 - Histogramme de distribution des prix
 - Graphique de répartition par catégorie
 - Comparaison des prix moyens par fournisseur
6. Exporte les données en CSV

3.1.2 Code de départ

```
1 import requests
2 from bs4 import BeautifulSoup
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import re
6 import time
7
8 # Configuration matplotlib pour accents
9 plt.rcParams['font.sans-serif'] = ['DejaVu Sans']
10
11 def extraire_prix(texte_prix):
12     """
13     Extrait le prix numerique d'une chaine
14     Ex: '95.50 euros/m3' -> 95.50
15     """
16     match = re.search(r'(\d+\.\d*)', texte_prix)
17     if match:
18         return float(match.group(1))
19     return 0.0
20
21 def compter_etoiles(texte_note):
22     """
```

```

23     Compte le nombre d'étoiles pleines
24     Ex: 5 étoiles pleines, 0 vides -> 5
25     """
26     # Compter les étoiles pleines (caractere Unicode U+2605)
27     return texte_note.count('\u2605')
28
29 def scraper_page(url):
30     """Scrape une page et retourne la liste des produits"""
31     response = requests.get(url)
32
33     if response.status_code != 200:
34         print(f"Erreur : {response.status_code}")
35         return []
36
37     soup = BeautifulSoup(response.text, 'html.parser')
38     produits = []
39
40     # Trouver tous les produits
41     cards = soup.find_all('div', class_='product-card')
42
43     for card in cards:
44         produit = {
45             'Type': '',
46             'Nom': '',
47             'Fournisseur': '',
48             'Prix': 0.0,
49             'Unite': '',
50             'Note': 0,
51             'Disponibilite': '',
52             'Delai': '',
53             'Region': ''
54         }
55
56         # A completer : extraire toutes les donnees
57         # Extraire le type
58         type_elem = card.find('span', class_='product-type')
59         if type_elem:
60             produit['Type'] = type_elem.text.strip()
61
62         # A completer : extraire les autres champs...
63
64         produits.append(produit)
65
66     return produits
67
68 def analyser_donnees(df):
69     """Analyse statistique des donnees"""
70     print("\n" + "="*60)
71     print("ANALYSE DES DONNEES - CATALOGUE MARKETBTP")
72     print("="*60)
73
74     # Informations generales
75     print(f"\nNombre total de produits : {len(df)}")
76     print(f"Nombre de categories : {df['Type'].unique()}")
77     print(f"Nombre de fournisseurs : {df['Fournisseur'].unique()}")
78
79     # Statistiques sur les prix
80     print("\n--- STATISTIQUES DES PRIX ---")
81     print(df['Prix'].describe())
82
83     # A completer : autres analyses...
84
85 def visualiser_donnees(df):

```

```

86     """Cree des graphiques"""
87
88     # 1. Top 10 des produits les plus chers
89     plt.figure(figsize=(12, 6))
90     top10 = df.nlargest(10, 'Prix')
91     plt.barh(range(len(top10)), top10['Prix'], color='steelblue')
92     plt.yticks(range(len(top10)), top10['Nom'], fontsize=9)
93     plt.xlabel('Prix (euros)', fontsize=12)
94     plt.title('Top 10 des produits les plus chers',
95              fontsize=14, fontweight='bold')
96     plt.tight_layout()
97     plt.savefig('top10_produits.png', dpi=300)
98     plt.show()
99
100    # A completer : autres graphiques...
101
102    # Programme principal
103    def main():
104        print("="*60)
105        print("COLLECTEUR ET ANALYSEUR MARKETBTP")
106        print("="*60)
107
108        base_url = 'http://www.malomatique.free.fr/MarketBTP/'
109        pages = ['index.html', 'page-2.html', 'page-3.html']
110        tous_les_produits = []
111
112        # Scraping des 3 pages
113        for i, page in enumerate(pages, 1):
114            url = base_url + page
115            print(f"\nScraping page {i}...")
116            produits = scraper_page(url)
117            tous_les_produits.extend(produits)
118            time.sleep(1) # Pause pour ne pas surcharger le serveur
119
120        # Conversion en DataFrame
121        df = pd.DataFrame(tous_les_produits)
122
123        print(f"\nTotal de produits collectes : {len(df)}")
124
125        # Nettoyage
126        df = df[df['Prix'] > 0]
127
128        # Analyse
129        analyser_donnees(df)
130
131        # Visualisation
132        visualiser_donnees(df)
133
134        # Export CSV
135        df.to_csv('marketbtp_analyse.csv', index=False, encoding='utf-8')
136        print("\nDonnees exportees dans 'marketbtp_analyse.csv'")
137
138    if __name__ == "__main__":
139        main()

```

Listing 8 – exercice1.py - Structure de base

3.1.3 Indices pour l'extraction

```

1 # Type de produit
2 type_elem = card.find('span', class_='product-type')
3 produit['Type'] = type_elem.text.strip() if type_elem else ''

```

```

4
5 # Nom du produit
6 nom_elem = card.find('h2', class_='product-name')
7 produit['Nom'] = nom_elem.text.strip() if nom_elem else ''
8
9 # Fournisseur (dans <p class="supplier">)
10 fournisseur_elem = card.find('p', class_='supplier')
11 if fournisseur_elem:
12     # Texte : "Fournisseur : Lafarge Holcim"
13     texte = fournisseur_elem.text.strip()
14     produit['Fournisseur'] = texte.replace('Fournisseur : ', '')
15
16 # Prix
17 prix_elem = card.find('div', class_='price')
18 if prix_elem:
19     texte_prix = prix_elem.text.strip()
20     produit['Prix'] = extraire_prix(texte_prix)
21     # Extraire l'unité
22     unite_elem = prix_elem.find('span', class_='unit')
23     if unite_elem:
24         produit['Unite'] = unite_elem.text.strip().replace('/', ' ')
25
26 # Note (etoiles)
27 note_elem = card.find('div', class_='rating')
28 if note_elem:
29     produit['Note'] = compter_etoiles(note_elem.text)
30
31 # Disponibilite
32 dispo_elem = card.find('span', class_='availability')
33 if dispo_elem:
34     produit['Disponibilite'] = dispo_elem.text.strip()
35
36 # Caracteristiques techniques (dans <div class="specs">)
37 specs_div = card.find('div', class_='specs')
38 if specs_div:
39     spec_items = specs_div.find_all('div', class_='spec-item')
40     for spec in spec_items:
41         label_elem = spec.find('span', class_='spec-label')
42         value_elem = spec.find('span', class_='spec-value')
43         if label_elem and value_elem:
44             label = label_elem.text.strip().replace(':', ' ')
45             value = value_elem.text.strip()
46             if 'Delai' in label:
47                 produit['Delai'] = value
48             elif 'Region' in label:
49                 produit['Region'] = value

```

Listing 9 – Indices pour extraire les données

3.2 Exercice 2 : Analyse comparative avancée (40 min)

3.2.1 Objectifs

À partir des données scrapées, réalisez une analyse comparative approfondie :

1. **Analyse par région** : Comparez les prix moyens entre les 3 régions
2. **Analyse temporelle** : Identifiez les produits avec les délais les plus courts/longs
3. **Rapport qualité/prix** : Corrélation entre note et prix
4. **Compétitivité des fournisseurs** : Qui offre les meilleurs prix ?
5. **Recommandations** : Générez un rapport avec les meilleures opportunités

3.2.2 Code de départ

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 # Charger les donnees du CSV genere
6 df = pd.read_csv('marketbtp_analyse.csv')
7
8 print("="*60)
9 print("ANALYSE COMPARATIVE AVANCEE")
10 print("="*60)
11
12 # 1. Analyse par region
13 print("\n--- PRIX MOYEN PAR REGION ---")
14 prix_region = df.groupby('Region')['Prix'].agg(['mean', 'median', 'count'])
15 print(prix_region)
16
17 # Graphique
18 plt.figure(figsize=(10, 6))
19 prix_region['mean'].plot(kind='bar', color='skyblue', edgecolor='black')
20 plt.title('Prix moyen par region', fontsize=14, fontweight='bold')
21 plt.ylabel('Prix moyen (euros)')
22 plt.xlabel('Region')
23 plt.xticks(rotation=45, ha='right')
24 plt.tight_layout()
25 plt.savefig('prix_par_region.png', dpi=300)
26 plt.show()
27
28 # 2. Rapport qualite/prix
29 print("\n--- RAPPORT QUALITE/PRIX ---")
30 # Calculer un score : Note / Prix (normalise)
31 df['Score_QP'] = df['Note'] / (df['Prix'] / df['Prix'].mean())
32
33 # Top 10 meilleurs rapports qualite/prix
34 top_qp = df.nlargest(10, 'Score_QP')[['Nom', 'Prix', 'Note', 'Score_QP']]
35 print("\nTop 10 meilleurs rapports qualite/prix :")
36 print(top_qp)
37
38 # 3. Opportunités (En stock + Prix < moyenne)
39 print("\n--- OPPORTUNITES ---")
40 prix_moyen = df['Prix'].mean()
41 opportunités = df[
42     (df['Disponibilite'] == 'En stock') &
43     (df['Prix'] < prix_moyen) &
44     (df['Note'] >= 4)
45 ].sort_values('Prix')
46
47 print(f"\n{len(opportunités)} opportunités detectees")
48 print(opportunités[['Type', 'Nom', 'Prix', 'Fournisseur']].head(10))
49
50 # Sauvegarder le rapport
51 opportunités.to_csv('opportunités_marketbtp.csv',
52     index=False, encoding='utf-8')
53 print("\nRapport sauvegarde dans 'opportunités_marketbtp.csv'")

```

Listing 10 – exercice2.py - Analyse avancee

4 Cas d'usage réels en Génie Civil avec Pandas

4.1 Analyse de prix de matériaux

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Donnees historiques de prix
5 df_prix = pd.DataFrame({
6     'Date': pd.date_range('2024-01-01', periods=12, freq='M'),
7     'Beton': [95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106],
8     'Acier': [850, 860, 855, 870, 880, 875, 890, 895, 900, 905, 910, 915]
9 })
10
11 # Calcul de l'evolution
12 df_prix['Beton_variation'] = df_prix['Beton'].pct_change() * 100
13 df_prix['Acier_variation'] = df_prix['Acier'].pct_change() * 100
14
15 # Moyenne mobile sur 3 mois
16 df_prix['Beton_MA3'] = df_prix['Beton'].rolling(window=3).mean()
17
18 # Visualisation
19 df_prix.plot(x='Date', y=['Beton', 'Acier'],
20             title='Evolution des prix des materiaux')
21 plt.ylabel('Prix (euros)')
22 plt.show()

```

Listing 11 – Analyse de series temporelles

4.2 Analyse d'appels d'offres

```

1 # Exemple de DataFrame d'appels d'offres
2 df_ao = pd.DataFrame({
3     'Reference': ['A0001', 'A0002', 'A0003', 'A0004'],
4     'Type': ['Batiment', 'Route', 'Pont', 'Batiment'],
5     'Montant': [500000, 1200000, 3000000, 750000],
6     'Duree_mois': [12, 18, 36, 15],
7     'Departement': ['75', '92', '93', '75']
8 })
9
10 # Analyses
11 montant_moyen_par_type = df_ao.groupby('Type')['Montant'].mean()
12 print(montant_moyen_par_type)
13
14 # Filtre : projets > 1M euros a Paris
15 grands_projets_paris = df_ao[
16     (df_ao['Montant'] > 1000000) &
17     (df_ao['Departement'] == '75')
18 ]

```

Listing 12 – Statistiques sur appels d'offres

5 Ressources complémentaires

5.1 Documentation officielle

- Pandas : <https://pandas.pydata.org/docs/>
- Matplotlib : <https://matplotlib.org/stable/contents.html>
- BeautifulSoup : <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- Requests : <https://requests.readthedocs.io/>

5.2 Tutoriels Pandas recommandés

- 10 minutes to Pandas (officiel)
- Pandas Cheat Sheet
- Real Python - Pandas tutorials

6 Points clés à retenir

6.1 Scraping

1. Toujours vérifier `robots.txt` et les CGU
2. Utiliser des délais entre requêtes (`time.sleep()`)
3. Gérer les erreurs avec `try/except`
4. Nettoyer les données extraites (strip, replace, etc.)

6.2 Pandas

1. Le DataFrame est la structure centrale
2. `.describe()` donne un aperçu statistique rapide
3. `.groupby()` permet des analyses par catégorie
4. `.plot()` offre des visualisations rapides
5. Toujours vérifier les types de données (`.dtypes`)

6.3 Analyse de données

1. Commencer par explorer les données (`.head()`, `.info()`)
2. Nettoyer avant d'analyser (valeurs manquantes, doublons)
3. Utiliser des graphiques pour comprendre les tendances
4. Documenter les sources et la méthodologie
5. Valider les résultats (cohérence, ordres de grandeur)

Workflow d'analyse de données BTP :

1. **Collecte** : Scraper les données du web
2. **Nettoyage** : Vérifier et corriger les données
3. **Structuration** : Organiser dans un DataFrame
4. **Analyse** : Calculer statistiques et tendances
5. **Visualisation** : Créer des graphiques parlants
6. **Export** : Sauvegarder résultats (CSV, Excel, PDF)
7. **Action** : Prendre des décisions basées sur les données

7 Préparation pour le TP2

Dans le TP2, vous allez développer un **Dashboard complet d'analyse de marché BTP** qui intègre :

- Un système automatisé de collecte de données
- Une analyse approfondie avec indicateurs avancés
- Des visualisations professionnelles multiples
- La génération automatique de rapports
- L'export multi-format (CSV, Excel, PDF)

Le TP2 sera l'occasion de consolider toutes les compétences acquises dans un projet complet et professionnel.

Fin du TD4 - Durée estimée : 2h30

Rendez-vous au TP2 pour le projet final !